九阶 · Linux 与 Python 自学资料

07

Docker 入门

把脚本打包成可交付的容器

教科书

AYMLiu-Pzw · 2026-10-22

九阶 · Linux 与 Python 自学资料

第 07 册

Docker 入门

把脚本打包成可交付的容器

资料类型	教科书
适用读者	完成 Python 主线 3-17 的学习者
发布日期	2026-10-22
作者署名	AYMLiu-Pzw

阅读导航

按章级标题定位正文；PDF 阅读器中的书签同时提供章、节两级入口。

1. 4-00 YAML 是什么
2. 开始之前 · 先把镜像加速器配好（只做一次）
3. 4-01 镜像与容器是什么：跑起第一个容器
4. 4-02 六件套：pull / run / ps / logs / exec / stop·rm
5. 4-03 数据卷与端口映射
6. 4-04 写第一个 Dockerfile
7. 4-05 把自己的脚本打包成镜像跑起来
8. 4-06 镜像瘦身与分层
9. 4-07 docker compose 起两个服务
10. 4-08 Compose 的网络与依赖顺序
11. 4-09 容器排障：日志 / 退出码 / 端口被占
12. 4-10 阶段 4 · 学习总结练习：三个脚本容器化 + README

07 • Docker 入门

本册是阶段 4 的正文，从 4-00 起步；配套与路线见《17-关卡总表》。
本册共 11 关（4-00 ~ 4-10）。目标：能起容器、挂数据卷、写 Dockerfile，并把自己的脚本打包成镜像。
本册是什么：Docker —— 把程序和依赖打成镜像，专治“在我机器上能跑”。
动手做实验时照这两条来：实验闭环（先记录原状态 → 操作 → 按输出 / 退出码验证 → 恢复现场）和踩坑记录位（现象 → 原因 → 解决 → 一句话总结）——不必逐关都写，做完整册回头总结一次就够。

4-00 YAML 是什么

为什么先学它：Docker Compose 和 K8s 的配置文件都是 YAML。不会读它，后面那两条命令你写不出来。
三句话说完：① **缩进 = 层级**（用空格，别用 Tab）；② **冒号 = 键和值**；③ 一行以 - 开头 = 列表里的一项。

```
services:            # 顶层的键
  web:               # 缩进 2 格：web 是 services 里的一项
    image: nginx     # 冒号左边是键，右边是值
    ports:
      - "8080:80"    # 前面带 - ：这是列表里的一项
```

怎么读：像读目录树——缩进越深，层级越里；冒号左边是名字，右边是内容。
三个坑：① 用 Tab 缩进会直接报错（YAML 只认空格）；② 冒号后面必须有一个空格；③ 同一个映射里不能出现重复键。不要依赖“后面的覆盖前面的”，不同解析器可能直接报错；重复键应视为配置错误。

完成判定

· 能说出上面那段里哪些是键、哪些是值 · 能用自己的话讲清「这一册学的东西是什么、干什么用的」（见本册开篇） · 能指出 "8080:80" 属于哪个键 · 能说出为什么不能用 Tab 缩进

开始之前 • 先把镜像加速器配好（只做一次）

为什么放在这里：下一步（4-01）你就要 `docker pull` 了。国内直连 Docker Hub 大概率超时，**先把这一步做掉**，后面 4-01、4-02、08 册的监控三件套、09 册的 K8s 才拉得动镜像。
不配会看到什么（真实报错样式）：

```
$ docker pull hello-world
Error response from daemon: Get "https://registry-1.docker.io/v2/": dial tcp 154.83.15.45:443: i/o timeout
```

看到 `i/o timeout` 就说明是"连不上仓库"，不是你的命令写错了。

三步配好（可直接复制）

```
$ sudo mkdir -p /etc/docker
$ if sudo test -e /etc/docker/daemon.json; then sudo cp -a /etc/docker/daemon.json /etc/docker/daemon.json.bak;
fi
$ sudo tee /etc/docker/daemon.json >/dev/null <<'EOF'
{
  "registry-mirrors": [
    "https://docker.m.daocloud.io",
    "https://docker.1ms.run"
  ]
}
EOF
$ sudo systemctl restart docker
$ docker info | grep -A3 "Registry Mirrors"
 Registry Mirrors:
  https://docker.m.daocloud.io/
  https://docker.1ms.run/
```

解读：`daemon.json` 是 Docker 的配置文件；`registry-mirrors` 是候选镜像加速源。**上面的覆盖写法只适合从未配置过 `daemon.json` 的个人实验机**；若文件已存在且包含 `data-root`、代理、日志驱动或私有仓库等键，必须先合并 JSON，不能直接用 `tee` 截断。命令已先备份到 `daemon.json.bak`，需要时可恢复。镜像加速源会随地区和服务策略变化。这里的两个地址是本次重测仍返回 registry 鉴权挑战的候选；registry 返回 401 通常表示入口存在并要求鉴权，403 或纯文本错误不能当作成功。最终以 `docker run --rm hello-world` 能否拉取为准，其他第三方地址只在现场验证可用后再加到候选列表，并避免优先级抖动；关键镜像还应记录 digest。

验证一次

```
$ docker run hello-world
Hello from Docker!
This message shows that your installation appears to be working correctly.
```

（注意区别：`docker pull` 只把镜像拉下来，输出是 `Using default tag / Pulling from / Digest / Status: Downloaded…` 这种状态行；让它"跑出那几行字"的是 `docker run`。）

三个坑：① JSON 里不能写注释，最后一个地址后面不能多逗号（多一个逗号 Docker 会起不来）；② 改完**必须** `sudo systemctl restart docker` 才生效；③ Docker 可能按列表尝试镜像源，但不保证“轮流成功”；某个源异常时先 `docker info` 核对配置，再更换可信源或直接使用上游。
三个都不行时（兜底）：① 换**阿里云容器镜像服务**的**个人专属**加速地址——登录阿里云 → 容器镜像服务（ACR）→「镜像加速器」页免费领取，按页面说明配置；② 或临时直连重试——`docker pull` 卡住就换个网络、或过一会儿再试；**自检**：`docker info` 的 Registry Mirrors 里是不是你配的那几个（改完没 `systemctl restart docker` 的话，这里还是旧值）。
只做一次。以后重装系统或换机器时，回来再做一遍。

4-01 镜像与容器是什么：跑起第一个容器

这一关只解决一件事：分清"镜像"和"容器"，并且亲手跑起来一个。
一个比喻：**镜像 = 菜谱**（只读的模板，放在仓库里）；**容器 = 照着菜谱做出来的那盘菜**（能跑、能停、能删，删了菜谱还在）。同一份菜谱可以同时做出好几盘菜（这就是"一个镜像起多个容器"）。

步骤 1：确认 Docker 装好了

```
$ docker --version
Docker version 29.1.3, build <本机构建号>   # 2026-10-21 的 Ubuntu 26.04 源快照；以 docker --version 实际输出为准
```

版本差怎么读：26.04 的 `docker.io` 是 29.x——`docker images` 的列可能变成 IMAGE / ID / DISK USAGE / CONTENT SIZE / EXTRA，`docker ps` / `docker port` 可能多出 IPv6 行。`docker build` 可能使用 BuildKit，也可能回退到经典构建器；**不要用固定的 Step 总数作完成标准**，看构建退出码和最终镜像能否运行。
解读：有版本号就是装好了。没装的话用 `sudo apt install -y docker.io`（Ubuntu 源里就有；**再装上 compose 插件**：`sudo apt install -y docker-compose-v2`，否则后面 4-07 起会报 **`docker: unknown command: docker compose`**）。
Docker 组权限边界：加入 `docker` 组后，用户可直接操纵 Docker daemon，而 daemon 以 root 运行，因此这**等同于授予接近 root 的权限**，可以挂载和修改宿主文件。**只限一次性个人实验机**执行 `sudo usermod -aG docker $USER` 后重新登录；共享服务器/生产机应保持使用 sudo，或采用 rootless Docker。不加组时若出现 **`permission denied ... /var/run/docker.sock`**，本套个人实验可按上面的权限边界处理，不要在共享环境照搬。

步骤 2：跑第一个容器

```
$ docker run hello-world

Hello from Docker!
This message shows that your installation appears to be working correctly.
...
```

解读：`docker run` 一共干了两件事——**先从仓库把镜像拉下来**（本地没有 hello-world 这个镜像），**再照着它起一个容器**跑出那几行字。容器跑完就退出了，镜像还留在你机器上。这就是"镜像和容器不是一回事"。

步骤 3：看清"镜像留下、容器退出"

```
$ docker images
REPOSITORY    TAG       IMAGE ID       CREATED       SIZE
hello-world   latest    ee301c921b8a   5 months ago  9.14kB

$ docker ps -a
CONTAINER ID   IMAGE         COMMAND    CREATED          STATUS                      NAMES
9c5b1e2a77f1   hello-world   "/hello"   10 seconds ago   Exited (0) 9 seconds ago    happy_allen
```

解读：`docker images` 列的是**镜像**（模板，只有一份）；`docker ps -a` 列的是**容器**（跑过的实例，**STATUS 那列写着 Exited (0)** = 正常退出，退出码 0）。`-a` 表示连已经退出的也一起看；不加 `-a` 只看正在跑的。

完成判定

· 能用自己的话说清镜像和容器的区别 · `docker run hello-world` 能跑出输出 · 能说出 `docker images` 和 `docker ps -a` 各看的是什么 · 能解释 STATUS 里的 Exited (0) 是什么意思
回滚方式：实验完把容器清掉即可——`docker rm happy_allen`（把 NAMES 换成你自己那行）；镜像想删就 `docker rmi hello-world`。删了都不影响系统。

4-02 六件套：pull / run / ps / logs / exec / stop•rm

这一关是：容器日常操作的六组命令（六节、七个命令名，最后一节 stop 和 rm 一起讲）。每条先按“**输入 → 预期输出 → 实际输出 → 解读**”走；第一次可以照打，过关要能撤掉教材完成下面迁移检查。

① pull：先把镜像拿下来

```
$ docker pull nginx:alpine
alpine: Pulling from library/nginx
...
Status: Downloaded newer image for nginx:alpine
```

解读：`nginx:alpine` 里的冒号后面是**标签（版本）**；alpine 是一个特别小的 Linux 发行版，用它做出来的镜像只有几十 MB。不写标签默认是 `latest`。

② run：起容器（最常用的两条参数）

```
$ docker run -d --name web -p 127.0.0.1:8080:80 nginx:alpine
7f3a9c1d2e4b8a6f0c5d3e2b1a9f8c7d6e5b4a3c2d1e0f9a8b7c6d5e4f3a2b1c
```

解读：`-d` = 后台跑（不占着你的终端）；`--name web` = 给容器起个名字（方便后面 stop/rm 时不用抄那串乱码 ID）；`-p 127.0.0.1:8080:80` = 只把**本机回环地址的 8080** 接到**容器 80**；最后的 `nginx:alpine` 是"用哪个镜像"。**输出那一长串就是容器 ID。**

③ ps：看现在有哪些容器

```
$ docker ps
CONTAINER ID   IMAGE          COMMAND                  STATUS         PORTS                    NAMES
7f3a9c1d2e4b   nginx:alpine   "/docker-entrypoint.…"   Up 12 seconds  127.0.0.1:8080->80/tcp   web
```

解读：**STATUS 是 Up 就说明它在跑**；PORTS 那列 `127.0.0.1:8080->80/tcp` 表示只在本机回环地址开放 8080，再转进容器 80。这时浏览器打开 `http://localhost:8080` 就能看到 nginx 欢迎页。**网卡边界**：若省略地址写成 `-p 8080:80`，Docker 默认发布到所有宿主网卡，输出会显示 `0.0.0.0:8080->80/tcp`；云主机或共享网络必须用安全组/防火墙限制，避免无意暴露。只供本机学习时保留 `-p 127.0.0.1:8080:80` 这类写法。（刚 run 的一两秒里页面可能还没就绪；`docker logs web` 里看到 "start worker processes" 就是稳了。）

④ logs：容器里的程序说了什么

```
$ docker logs web
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/10/09 12:03:11 [notice] 1#1: start worker processes
```

解读：容器里的程序输出不会留在你的终端里，要用 `logs` 捞出来——**排障第一步永远是看日志。**加 `-f` 就是实时盯（和 Linux 的 `tail -f` 一个意思）。

⑤ exec：进容器里面看

```
$ docker exec -it web sh
/ # ls /etc/nginx
conf.d          mime.types      nginx.conf    # 示例：条目数随 nginx 版本
/ # exit
```

解读：`-it` = 交互式终端（i 是标准输入，t 是终端）；`sh` 是要执行的命令（alpine 里没有 bash，用 sh）。**进容器只为了看和查，改在容器可写层里的内容会随容器删除**；需要保留的数据应放进数据卷或挂载目录。

⑥ stop / rm：停掉、删掉

```
$ docker stop web && docker rm web
web
web
```

解读：`stop` 是停止容器进程（先发 TERM，和 Linux 优雅停机同理），`rm` 才是删除容器。**停止但未删除的容器仍保留在 `docker ps -a` 中，继续占用容器名、配置和可能的磁盘空间；只有运行中的容器才占用已发布的宿主端口，停止后端口会释放。删除容器才去掉容器名，并释放它的可写层与元数据。**

完成判定

· 六节里的命令各自能不看笔记敲一遍 · 能说出 `-d`、`--name`、`-p` 各解决什么问题 · 能说出 `logs` 和 `exec` 分别什么时候用 · 能区分 stop 后释放端口与 rm 后清理容器名/磁盘 · 能独立完成 4-02 迁移检查（换回环端口并预测；错误标签从 run/pull 输出定位） · 练习完把容器清干净（`docker ps -a` 里没有遗留）

回滚方式：`docker stop` + `docker rm` 一删就回到原样；镜像用 `docker rmi nginx:alpine` 删（不吃磁盘时不用删）。

4-02 迁移检查（不先看命令答案）：① 合上书，说清 `run -d`、`logs`、`exec` 各自解决什么问题；② 把示例端口 `127.0.0.1:8080:80` 改成 `127.0.0.1:18080:80`，先预测浏览器地址、`docker port` 输出和容器名单，再实际验证；随后 `docker stop web`，确认 `docker ps -a` 仍能看到 web，但 `docker port web` 已无输出（端口释放）；③ 故意把镜像标签写错，例如运行 `docker run --rm nginx:alpne`，先记录 `docker run` 的拉取错误；再单独执行 `docker pull nginx:alpne`，根据 `manifest unknown` / `pull access denied` 等返回判断标签或鉴权问题。标签不存在时通常不会创建容器，所以不要用空的 `docker ps -a` / `docker logs` 推断根因。第③步只交现象、判断和结果，不抄完整答案。

4-03 数据卷与端口映射

这一关解决两个"为什么"：为什么容器里的数据一删就没？为什么外面连不上容器里的服务？

一、先把"数据会消失"亲眼看一遍

```
$ docker run -it --name tmp ubuntu:26.04 bash
root@5a1b2c3d4e5f:/# echo "hello" > /data.txt && cat /data.txt
hello
root@5a1b2c3d4e5f:/# exit

$ docker rm tmp && docker run -it --name tmp2 ubuntu:26.04 bash
root@9f8e7d6c5b4a:/# cat /data.txt
cat: /data.txt: No such file or directory
```

解读：容器是"一次性的盘子"——**删掉容器，写在容器里的东西全没了**，因为那些写入只存在于容器那一层可写层里。想留下，就得把数据放到容器外面。

二、数据卷 -v：把本机目录接进容器

```
$ mkdir -p ~/lab/docker-data && echo "我在外面写的" > ~/lab/docker-data/note.txt
$ docker run --rm -it -v ~/lab/docker-data:/data ubuntu:26.04 bash
root@a1b2c3d4e5f6:/# cat /data/note.txt
我在外面写的
root@a1b2c3d4e5f6:/# echo "我在容器里写的" >> /data/note.txt && exit

$ cat ~/lab/docker-data/note.txt
我在外面写的
我在容器里写的
```

解读：`-v 本机路径:容器路径` = 把本机的目录"接"到容器里。两边看到的是**同一份文件**：容器里改的，外面立刻能看到。这样容器删了，数据还在（因为文件本来就住在你机器上）。`--rm` 表示容器退出后自动删掉。

一句话记住：**无状态的放容器里，有状态的（数据、日志、配置）挂出来。**

三、端口映射 -p：让外面能连进来

```
$ docker run -d --name web -p 127.0.0.1:8080:80 nginx:alpine
$ curl -s -o /dev/null -w "%{http_code}\n" http://localhost:8080
200
$ docker port web
80/tcp -> 127.0.0.1:8080
```

解读：容器有自己的"内网"（里面 80 端口在听），外面默认连不上。`-p 127.0.0.1:8080:80` = **仅本机回环地址 8080 → 容器 80**（前面是本机地址和端口，后面是容器端口，别写反）。`curl` 返回 200 就说明通了。若写成 `-p 8080:80`，Docker 默认发布到所有宿主网卡；远程访问应另做防火墙、私网或 VPN 边界，不能只看到 localhost 能开就认为外部不可达。

完成判定

· 能说出"为什么容器删了数据就没了"，并演示一次数据卷保住数据 · 能自己起一个 nginx 并用 curl 验证 200 · 能说出 `-p 8080:80` 里前后两个数字各是什么 · 能说出什么该挂卷、什么不用

回滚方式：实验容器 `docker rm -f` 掉；挂载目录在 `~/lab` 里，删掉不影响系统。

4-04 写第一个 Dockerfile

Dockerfile 是什么：一份"怎么把东西装进镜像"的说明书，五行就能跑通。你只要先记住这五个指令，别的等用到再查。

指令	干什么	一句话理解
FROM	选基础镜像（从哪个系统开始搭）	打地基，永远写第一行
RUN	在**构建时**执行命令（装软件、建目录）	装修：装的时候做一次
COPY	把本机文件复制进镜像	把你的脚本搬进去
EXPOSE	声明容器里要听哪个端口（**给读的人看的说明**）	留个纸条：我用 80
CMD	容器**启动时**默认执行的那条命令	开火：跑起来时做

动手：写一个自己的 hello 镜像

```
$ mkdir -p ~/lab/myimage && cd ~/lab/myimage
$ cat > Dockerfile <<'EOF'
FROM ubuntu:26.04
RUN apt-get update && apt-get install -y curl
COPY hello.sh /hello.sh
RUN chmod +x /hello.sh
CMD ["/hello.sh"]
EOF
$ cat > hello.sh <<'EOF'
#!/bin/bash
echo "容器里跑起来了，里面装的是："
curl --version | head -1
EOF
```

构建并运行

```
$ docker build -t my-hello:1.0 .
[+] Building 12.4s
 => [internal] load build definition from Dockerfile
 => [build] FROM docker.io/library/ubuntu:26.04
 => [build] RUN apt-get update && apt-get install -y curl
 => [build] COPY hello.sh /hello.sh
 => [build] RUN chmod +x /hello.sh
 => exporting to image

$ docker run --rm my-hello:1.0
容器里跑起来了，里面装的是：
curl 8.5.0 (x86_64-pc-linux-gnu)  # 示例（版本号随机器）；脚本里 head -1 只取了第一行，完整输出还有一大串特性开关
```

解读：`docker build -t 名字:版本 .` 里的**最后那个点不能少**——它表示"用当前目录里的 Dockerfile"。示例 Dockerfile 有 **FROM、RUN、COPY、RUN、CMD 五条指令**；构建器会按顺序执行并缓存，但 BuildKit/经典构建器显示的分组数量不一定等于 Dockerfile 行数，**不要拿 [1/4] 之类的固定总数判对错**。构建退出码为 0 且容器能运行，才算通过。
为什么 COPY 要放在装软件之后：Docker 有缓存，**哪一层变了，它和它后面的层才重建**。你改脚本的时候不想重装一遍 curl，所以"变化少的放前面、变化多的放后面"。

完成判定

· 能不看笔记写出 FROM / RUN / COPY / CMD 四类指令（本关示例实际有 5 条指令） · 能说清 RUN 和 CMD 的区别（构建时 vs 启动时） · 能说清 build 命令最后那个点是什么 · 能说出 COPY 为什么要放在后面
回滚方式：练废的镜像 `docker rmi my-hello:1.0` 删掉重来；`~/lab/myimage` 里的文件随时可改。

4-05 把自己的脚本打包成镜像跑起来

这一关的意义：从"会跑别人的镜像"变成"**能把自己的东西装进镜像**"。这也是阶段 4 检查点的前置动作。

步骤 1：把脚本准备好

```
$ mkdir -p ~/lab/docker-scripts && cd ~/lab/docker-scripts
$ cp ~/lab/shell-scripts/disk_alert.sh .    # 换成你自己的脚本路径
$ cat disk_alert.sh
#!/bin/bash
THRESHOLD=80
USE=$(df -h / | awk 'NR==2 {print $5}' | tr -d '%')
if [ "$USE" -ge "$THRESHOLD" ]; then
  echo "[告警] 磁盘使用率 ${USE}%，超过阈值 ${THRESHOLD}%"
else
  echo "[正常] 磁盘使用率 ${USE}%"
fi
```

步骤 2：写 Dockerfile

```
FROM ubuntu:26.04
RUN apt-get update && apt-get install -y bash coreutils && rm -rf /var/lib/apt/lists/*
WORKDIR /opt/scripts
COPY disk_alert.sh ./disk_alert.sh
RUN chmod +x disk_alert.sh
CMD ["./disk_alert.sh"]
```

解读：`WORKDIR` = 进容器后默认站在哪个目录（省得每句都写绝对路径）；`rm -rf /var/lib/apt/lists/*` 是装完就删缓存，让镜像小一点；`CMD` 用方括号写法（推荐，避免 shell 解析带来的怪问题）。

步骤 3：构建、运行、验证

```
$ docker build -t disk-alert:1.0 .
$ docker run --rm disk-alert:1.0
[正常] 磁盘使用率 42%      # 示例数值，随你机器

$ docker run --rm -e THRESHOLD=10 disk-alert:1.0
[正常] 磁盘使用率 42%
```

解读：脚本在容器里跑通了。**注意后一条**：我们想试试改阈值，但脚本里阈值是写死的（THRESHOLD=80），所以在外面改环境变量没用。**这就是"参数要能传进去"的意义**——顺手把脚本改成从环境变量读阈值：

版本差异说明：本节贴的"写死版"是为了展示"参数要能传进去"的演进；你阶段 2 交付的 disk_alert.sh（03 册附录 D2）本来就是 THRESHOLD=${THRESHOLD:-80} 环境变量版——如果手头就是那份，可以跳过这步改写，直接 build 后用 -e 传值验证。

```
THRESHOLD=${THRESHOLD:-80}     # 有传就用传进来的，没传默认 80
```

改完重新 build 再跑，换一个一定会触发的值就能看到告警：`docker run --rm -e THRESHOLD=0 disk-alert:1.0`（或填一个低于你机器当前磁盘使用率的值）——**用 10 这类值在自己机器上可能根本不触发，那是"阈值没到"，不是"没生效"。这一步做出来，你就完成了"脚本 → 镜像 → 可配置服务"的升级。**

完成判定

· 自己的脚本能在容器里跑出结果 · 能说出 WORKDIR 解决了什么问题 · 改一处脚本后重新 build，能说清"哪几层用了缓存" · 用 -e 把阈值传进去并触发一次告警

回滚方式：`docker rmi disk-alert:1.0` 删镜像；脚本原件在 ~/lab 和 shell-scripts 仓库里，没被改动过。

顺带一提 · SQL（不用学，知道有这么回事就行）

是什么：跟数据库说话的语言，最常用的四句是 SELECT（查）、INSERT（插）、UPDATE（改）、DELETE（删）。
什么时候遇到：容器化部署里常带一个数据库容器；工单里会有"数据库连不上""连接数满了""这条数据查不到"。
真要用时去哪查：14 手册的排错心法（先看容器状态与日志）+ 搜"mysql 连接数 查看"，四句话的语法一看就懂。

4-06 镜像瘦身与分层

这一关只解决一件事：让你的镜像不要又大又慢。你要是只会写 Dockerfile，镜像是能跑，但别人问"你这镜像多大、为什么这么大"，就答不上来了。

步骤 1：先看看镜像是怎么做出来的

```
$ docker history my-hello:1.0
IMAGE          CREATED BY                                      SIZE
9f8e7d6c5b4a   CMD ["/hello.sh"]                               0B
1a2b3c4d5e6f   RUN chmod +x /hello.sh                          0B
2b3c4d5e6f70   COPY hello.sh /hello.sh                         121B
3c4d5e6f7081   RUN apt-get update && apt-get install -y curl   92MB
4d5e6f708192   /bin/sh -c #(nop)  FROM ubuntu:26.04            78MB
```

解读：Dockerfile 里**每一条指令就是一层**，从下往上看就是构建顺序。**装 curl 那一条一口气吃掉 92MB** —— 因为 apt 装的包加上缓存全留在这一层里了。

步骤 2：把体积压回去

```
FROM ubuntu:26.04
RUN apt-get update && apt-get install -y curl \
    && rm -rf /var/lib/apt/lists/*          # ← 装完在同一层里删缓存
COPY hello.sh /hello.sh
RUN chmod +x /hello.sh
CMD ["/hello.sh"]
```

```
$ docker build -t my-hello:1.1 . && docker images | head -3
REPOSITORY   TAG   IMAGE ID       SIZE
my-hello     1.1   7a8b9c0d1e2f   96MB
my-hello     1.0   9f8e7d6c5b4a   170MB
```

解读：**170MB → 96MB。**关键动作只有一个：**"装东西"和"清缓存"要写在同一层里**（用 && 连起来）。分成两条 RUN，中间那层已经把缓存固化下来了，后面删也没用。

步骤 3：缓存与构建顺序（观察层是否复用）

```
$ docker build -t my-hello:1.2 .      # 什么都没改
 => CACHED RUN apt-get update && apt-get install -y curl ...
 => CACHED COPY hello.sh /hello.sh
# 改一行 hello.sh 再 build：
 => CACHED RUN apt-get update ...              ← 没重装 curl
 => COPY hello.sh /hello.sh                    ← 从这层开始重做
```

解读：**哪一层变了，它和它后面的层才重建。**"很少变的东西放前面，经常变的东西放后面" 就是这个道理（所以装依赖在 COPY 代码之前）。

延伸（点到为止）：多阶段构建

```
# 第一阶段：编译用的“工地”
FROM golang:1.23 AS builder
WORKDIR /src
COPY main.go .
RUN go build -o app main.go

# 第二阶段：只把成品搬进小房子
FROM alpine:3.20
COPY --from=builder /src/app /app
CMD ["/app"]
```

解读：把"编译要的一大堆工具"留在第一阶段，**最终镜像里只放能跑的成品。**本段只示范语法，`golang:1.23` 与 `alpine:3.20` 不是 2026-10 的“最新基线”；真正用于生产时先查当前受支持版本、固定 tag/digest 并复测。

完成判定

· 能用 `docker history` 说出哪一层最大 · 把"装 + 清缓存"写进同一层后，镜像确实变小 · 能说出为什么 COPY 要放在装依赖之后 · 能说出多阶段构建解决了什么问题（一句话）

回滚方式：旧镜像 `docker rmi my-hello:1.0` 删掉即可；Dockerfile 改坏了就从 `~/lab/myimage` 里的备份恢复。

4-07 docker compose 起两个服务

为什么需要 compose：一条 `docker run` 只能起一个容器。真实场景动辄"应用 + 缓存 + 数据库"三四个容器，一个个手敲容易漏参数。**compose 就是把这些 `docker run` 写成一份文件**，一声令下全起来。

步骤 1：写文件

```
$ mkdir -p ~/lab/two-services && cd ~/lab/two-services
$ cat > docker-compose.yml <<'EOF'
services:
  web:
    image: nginx:alpine
    container_name: web
    ports:
      - "127.0.0.1:8080:80"
    depends_on:
      - cache
    restart: unless-stopped

  cache:
    image: redis:7-alpine
    container_name: cache
    restart: unless-stopped
EOF
```

解读：这份文件就是两个 `docker run` 的"清单版"。注意 `services:` 下缩进的两格才是服务名（**YAML 的缩进就是层级**，4-00 学过）。depends_on 表示"起 web 之前先起 cache"。

步骤 2：起来

```
$ docker compose up -d
[+] Running 3/3
 ✓ Network two-services_default   Created
 ✓ Container cache                Started
 ✓ Container web                  Started

$ docker compose ps
NAME    IMAGE              STATUS         PORTS
cache   redis:7-alpine     Up 10 seconds  6379/tcp
web     nginx:alpine       Up 10 seconds  127.0.0.1:8080->80/tcp
```

解读：**两个都是 Up 才算成功。**compose 还顺带建了一个网络（`two-services_default`），下一关讲它。`-d` 还是"后台跑"的意思。

步骤 3：验证两个服务真的在

```
$ curl -s -o /dev/null -w "%{http_code}\n" http://localhost:8080
200
$ docker compose exec cache redis-cli ping
PONG
$ docker compose logs web | tail -2
2026/10/09 12:30:01 [notice] 1#1: start worker process 29
```

解读：**200 = web 活着，PONG = cache 活着。**注意命令从 `docker exec` 变成了 `docker compose exec 服务名` —— 用服务名就不用抄容器 ID 了。

步骤 4：收工

```
$ docker compose down
[+] Running 3/3
 ✓ Container web    Removed
 ✓ Container cache  Removed
 ✓ Network two-services_default  Removed
```

解读：down 一条命令把容器和网络都清掉（**但具名卷不会删**，这是有意的：数据别跟着容器一起消失）。

完成判定

· 能不看笔记写出一个两服务的 compose 文件 · up -d / ps / logs / exec / down 五条命令能跑通 · 能用 curl 200 和 redis PONG 各验证一次 · 能说出 down 之后什么被删了、什么没被删

回滚方式：`docker compose down`；想连数据卷一起清就 `down -v`（练习环境随便清，生产环境别乱加 -v）。

4-08 Compose 的网络与依赖顺序

一、服务之间靠"名字"互相找到

```
$ cd ~/lab/two-services && docker compose up -d
$ docker compose exec web ping -c 2 cache
PING cache (172.20.0.3): 56 data bytes
64 bytes from 172.20.0.3: seq=0 ttl=64 time=0.071 ms

$ docker compose exec web ping -c 1 baidu.com
ping: bad address 'baidu.com'          # ← 如果这里显示的是 IP（能解析），说明容器用的是宿主机 DNS，正常；解析不通再查 daemon 的 DNS 配置
```

解读：compose 给这组服务建了**一个专属网络**，并在里面做了 DNS：**服务名（web / cache）就是主机名。**所以代码里连 Redis 就写 `redis://cache:6379`，而不是 `localhost`——对 web 容器来说，localhost 是它自己。

这就是 5-02 里 Prometheus 要写 `node-exporter:9100` 的同一个道理。

二、depends_on 不等于"对方已经准备好了"

```
# 现象：web 启动了，但连不上 redis，报连接被拒
$ docker compose logs web | grep -i "connection" | head -2
 Connection refused (cache:6379)

# 原因：depends_on 只管"先启动 cache 容器"，
#       不代表 redis 进程已经能接受连接（它可能还在加载数据）
```

解读：depends_on 保证的是**启动顺序**，不是**就绪状态。**这是实战里的经典坑。两种正经解法：

```
# 解法 A（推荐）：应用侧重试 —— 连不上就等 2 秒再试，最多 10 次
# 解法 B：给 cache 加健康检查，web 等它"健康"再起
services:
  cache:
    image: redis:7-alpine
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 5s
      retries: 5
  web:
    image: nginx:alpine
    depends_on:
      cache:
        condition: service_healthy      # ← 等它健康，而不是"启动了"
```

解读：解法 A 更常用（应用自己扛住"依赖没就绪"），解法 B 更漂亮但只有 compose 支持。两个都记住，够用了。

三、看一眼网络长什么样

```
$ docker network ls | grep two
8f1c2d3e4f5a   two-services_default   bridge    local
$ docker network inspect two-services_default --format '{{range .Containers}}{{.Name}} {{end}}'
web cache
```

解读：同一个 compose 项目里的服务自动进同一个 bridge 网络，**不同项目之间默认互相看不见**（这也是隔离）。

完成判定

· 能说出为什么代码里写 `cache:6379` 而不是 `localhost:6379` · 能说出 depends_on 保证了什么、没保证什么 · 能用两种解法之一演示"依赖没就绪时不崩" · 能用 `docker network inspect` 说出哪个网络里有哪几个容器

回滚方式：`docker compose down`；实验用的网络会跟着一起删。

4-09 容器排障：日志 / 退出码 / 端口被占

这一关是阶段 4 的重点：容器起不来时，你要能在 3 分钟内说清"是哪一类问题"。下面三条是排查链，不是命令清单 —— **每一条都是"现象 → 命令 → 判读 → 下一步"。**

排查链 ①：容器起不来，先看日志

```
$ docker compose up -d          # 结果 web 没稳住
[+] Running 2/2
 ✓ Container cache  Started
 ✓ Container web    Started          ← 注意：这里不是 Error

$ docker compose ps             # ← 先看状态：Restarting = 起来又自己退了
web   …   Restarting (1) 3 seconds ago

$ docker compose logs web       # ← 第一步永远是日志
web  | nginx: [emerg] invalid number of arguments in "listen" directive in /etc/nginx/conf.d/default.conf:3
```

判读：**"Started" 不等于跑住了**——4-07 的 compose 配了 `restart: unless-stopped`，容器退出后会被自动拉起，所以**看不到 Error**，而是 `Restarting` 循环；日志直接告诉你是**配置写错**（nginx 的 listen 少写了端口）。
下一步：改配置 → `docker compose up -d` 重来。**"Started" 或 "Restarting" 时都先 `docker compose ps` 看状态、再看日志。**若日志是空的，走第 ② 条。

排查链 ②：日志空的，就看退出码

```
$ docker ps -a | grep web
7f3a9c1d2e4b   web   nginx:alpine   Exited (1) 2 minutes ago

$ docker inspect web --format '{{.State.ExitCode}} {{.State.OOMKilled}}'
1 false
```

退出码	常见含义	下一步看哪
0	正常结束（一次性任务的正常态）	不是故障，看它该不该一直跑
1	应用自己报错退出	日志；检查配置 / 依赖 / 参数
137	被 SIGKILL 杀掉，最常见是**内存超限被 OOM**	`docker inspect` 看 `OOMKilled`；加大内存或查内存泄漏
143	进程自己处理 SIGTERM 后主动退出（**128 + 15**；**不是所有 `docker stop` 都是 143**）	看应用怎么处理 SIGTERM：优雅退出 = 0 / 被强杀 = 137

判读：`137` + `OOMKilled: true` = 内存超限被 OOM；`137` 但没 OOM = 被 SIGKILL（最常见就是 `docker stop` 发了 SIGTERM、等 10 秒没停住，然后强杀），`0` = 应用处理 SIGTERM 后优雅退出。
实测（三组，可自己复现）：`docker stop` 一个 `alpine sleep 300` → **137**（等满 10 秒后强杀）；`docker stop` 一个 `nginx:alpine` → **0**（1 秒内优雅退出；**前提是让它先跑起来再停**——刚 run 就秒停会得到 137，那是没来得及处理信号）；`docker kill` → **137。所以别背「stop 就是 143」——要看应用怎么处理 SIGTERM。**
日志里还有一类"没见过"的报错：`exec format error`——意思是"可执行文件格式不对"。容器里最常见两因：① 脚本第一行 shebang 缺失或写错（如漏了 `#!/bin/bash`）；② 文件在 Windows 里编辑过、带了 CRLF 换行。修法：`head -1` 看第一行、对齐换行符，再 `chmod +x` 后重新 build 跑。

排查链 ③：容器在跑，但外面连不上（端口被占）

```
$ docker compose up -d
docker: Error response from daemon: failed to set up container networking: driver failed
programming external connectivity on endpoint web2 (…): Bind for 0.0.0.0:8080 failed: port is already allocated

$ ss -tuln | grep 8080          # 先确认端口已被监听，不读取其他用户的进程信息
tcp   LISTEN 0 511  0.0.0.0:8080  0.0.0.0:*

$ sudo ss -tulnp | grep 8080    # 再看 PID/进程：普通用户通常看不到 root 进程
tcp   LISTEN 0 511  0.0.0.0:8080  0.0.0.0:*  users:(("nginx",pid=2211,fd=6))

$ docker ps --format '{{.Names}}\t{{.Ports}}' | grep 8080
old-nginx   0.0.0.0:8080->80/tcp
```

判读：**本机 8080 端口已经被占了**（要么另一个容器，要么一个直接跑在系统上的 nginx）。
下一步：二选一 —— 把新服务换到别的端口（`-p 8081:80`），或者把占用的那位停掉（`docker stop old-nginx` / `systemctl stop nginx`）。**先确认那个服务是不是别人正在用的，再动手停。**

三条链合起来：一张排查顺序图

```
容器没起来
 ├─ ① docker compose logs 服务名    → 有报错：按报错改（配置/依赖/参数）
 ├─ ② docker ps -a 看 Exited (N)    → 137/OOMKilled：内存问题
 │                                    143：被谁停了
 │                                    1：应用内部报错，回 ①
 └─ ③ 容器 Up 但连不上              → ss -tulnp 看端口 → 被占就换端口或停掉占用者
```

完成判定

· 能不看笔记走完三条排查链 · 能说出 137 和 143 各代表什么 · 能故意造一个"端口被占"并解决掉 · 能说出"容器 Up 但连不上"时先用 `ss -tuln` 判断占用，再看所有者时用 `sudo ss -tulnp`

回滚方式：练习造的故障容器全部 `docker rm -f` 清掉；改动过的端口配置改回原样。

4-10 阶段 4 • 学习总结练习：三个脚本容器化 + README

业务场景

你阶段 2 写的三个脚本（backup / disk_alert / service_guard）现在只躺在你的机器上。同事说："换台机器就得重配一遍环境，能不能打包成镜像，谁拿去都能跑？"

容器化范围：就用阶段 2 的 `backup.sh` / `disk_alert.sh` / `service_guard.sh` 这三个（5 件套里的其余两个 check_res / health_check 是配套脚本，不必容器化）。

综合要求（这题把这几关的技能串起来）

技能点：**Dockerfile 分层与缓存（4-04/4-06） + 环境变量传参（4-05） + 数据卷（4-03） + compose 两个服务（4-07/4-08） + 容器排障（4-09） + README 写法（16 册那一页）**

① 给三个脚本各写一个 Dockerfile，**装依赖和清缓存写在同一层**；

② 阈值一类的参数**从环境变量读**（`${THRESHOLD:-80}` 那种写法），不要写死；

③ 用一个 `docker-compose.yml` 把"巡检脚本 + 一个服务"两个容器一起管起来（服务可以是 nginx，模拟被巡检的对象）；

④ 把巡检结果**写到挂载出来的目录**里（容器删了结果还在）；

⑤ 写 README：一句话说明 / 怎么跑 / 环境 / 效果输出 / 踩坑记录（照 16 册《一个合格的 README 长什么样》）。

验收

· 三个镜像都能 build 成功，且 `docker history` 里没有"又大又不该留"的层 · 改阈值用 `-e` 传进去能触发告警 · compose 两个服务都是 Up · 报告文件在宿主机目录里能找到 · 陌生人照 README 能跑通（写法见 16 册《一个合格的 README 长什么样》） · 每一条都能说清"为什么这么做"

产出

仓库 `docker-scripts`（或并进 `shell-scripts` 的 `docker/` 目录）：三个 Dockerfile + compose 文件 + README + 一张运行截图。

AI 运维指向

这是"把运维脚本变成可分发工具"的标准动作 —— 以后你把推理服务打成镜像、在 K8s 里跑，用的是同一套思路。

本册复习卡

① 镜像 = 构建好的只读模板；容器 = 运行实例；卷 = 留在容器生命周期之外的数据。

② 容器没起来先看 `docker compose logs` 与 `ps -a`；Up 但访问不到再看端口映射与监听。

③ 独立变式：把自己的一行脚本做成镜像，换一个端口运行；先写下预计的 `docker port` 与 `docker ps` 状态，再做并解释差异。正文不附这条变式的完整答案。